FR

Bienvenue sur Sakata

Ce portail est un sanctuaire numérique dédié à la préservation et à la transmission de l'identité culturelle du peuple **Sakata**.

Notre objectif est d'unir nos racines ancestrales avec la splendeur numérique moderne, en offrant à la fois des archives vivantes, des sagesses intemporelles et un forum d'échange intergénérationnel.

Suivant →

> *Le cache est l'arme la plus efficace contre la latence — et la source la plus sournoise de bugs. Sakata utilise trois couches complémentaires, chacune avec ses règles d'invalidation et son TTL.*

VUE D'ENSEMBLE

Les trois couches

COUCHE	LIEU	TTL TYPIQUE	CAS D'USAGE
1. localStorage	Navigateur	24h	Articles vus, profils consultés, paramètres user
2. ISR (Cache-Control)	Edge Netlify + CDN	5 min + SWR 60s	API routes /api/articles, /api/profiles
3. SWR (useCachedFetch)	Mémoire React	Session	Composants qui re-fetchent au focus

COMBINÉES

Une page typique utilise les trois : SWR en mémoire pour la vivacité, ISR pour soulager Supabase, localStorage comme filet offline et persistance entre sessions.

PERSISTANCE NAVIGATEUR

Couche 1 — localStorage

Règles absolues

- Toute clé DOIT commencer par `sakata-`.
- Toute clé DOIT être déclarée dans `storage.ts`.
- Toute clé DOIT être ajoutée à `SAKATA_KEY_WHITELIST`.
- Pas de hardcoding — toujours via constante ou helper.
- Stocker un `timestamp` avec la valeur pour TTL.

Format standard

CODE TYPESCRIPT

```typescript
type CachedEntry<T> = {
  data: T;
  cachedAt: number;
  version: string;  // APP_VERSION
};

function setCache<T>(key: string, data: T) {
  localStorage.setItem(key, JSON.stringify({
    data,
    cachedAt: Date.now(),
    version: APP_VERSION,
  }));
}

function getCache<T>(key: string, maxAgeMs: number): T | null {
  const raw = localStorage.getItem(key);
  if (!raw) return null;
  const entry: CachedEntry<T> = JSON.parse(raw);
  if (entry.version !== APP_VERSION) return null;
  if (Date.now() - entry.cachedAt > maxAgeMs) return null;
  return entry.data;
}
```

Versioning APP_VERSION

Quand on bumpe `APP_VERSION` dans `business.ts`, l'AuthProvider parcourt toutes les clés `sakata-*` et purge celles qui n'ont pas la nouvelle version. Évite les incompatibilités après refonte de schéma.

EDGE CACHE

Couche 2 — ISR (Cache-Control headers)

Pattern API route

SRC/APP/API/ARTICLES/ROUTE.TS TYPESCRIPT

```typescript
export async function GET() {
  const articles = await fetchArticles();

  return NextResponse.json(articles, {
    headers: {
      "Cache-Control": "public, s-maxage=300, stale-while-revalidate=60",
      // s-maxage = 5 min CDN cache
      // stale-while-revalidate = sert l'ancien pendant 60s pendant qu'on regenere
    },
  });
}
```

Routes avec cache ISR

- `/api/articles` — liste d'articles (5 min)
- `/api/profiles` — annuaire membres (10 min)
- `/api/courses` — contenu école (1h)
- `/api/translations` — chaînes i18n (1h)

NE JAMAIS CACHER

- Routes user-spécifiques (`/api/me`, profil perso).
- Routes admin (`/api/admin/*`).
- Routes de mutation (POST, PATCH, DELETE).
- Routes de webhook (Stripe).

MÉMOIRE COMPOSANT

Couche 3 — SWR via useCachedFetch

Le hook

CODE TYPESCRIPT

```typescript
const { data, error, isLoading, mutate } = useCachedFetch<Article[]>({
  key: "articles-list",
  fetcher: () => supabase.from("articles").select(),
  ttlMs: 5 * 60 * 1000,          // 5 min
  revalidateOnFocus: true,         // Refetch au retour onglet
  revalidateOnReconnect: true,     // Refetch après coupure réseau
});
```

Stratégie stale-while-revalidate

1. Vérifier le localStorage → si valide (TTL OK), retourner immédiatement.
2. Lancer un fetch en arrière-plan (silencieux).
3. Si la nouvelle data diffère, mettre à jour state + localStorage.
4. L'utilisateur voit le contenu instantanément, et la fraîcheur arrive en quelques ms.

MAINTENANCE

Patterns d'invalidation

Invalider après mutation

CODE TYPESCRIPT

```typescript
async function publishArticle(id: string) {
  await supabase.from("articles").update({ status: "published" }).eq("id", id);

  // 1. Invalider le cache local
  mutate(undefined, { revalidate: true });

  // 2. Invalider le cache CDN (Next.js revalidatePath)
  await fetch("/api/revalidate?path=/savoir", { method: "POST" });

  // 3. Le cache localStorage des autres clients sera périmé naturellement (TTL)
}
```

Invalidation cross-tab

Pour propager une invalidation entre onglets (ex: user déconnecté dans un onglet, doit l'être partout), utiliser `BroadcastChannel`.

ERREURS COURANTES

Anti-patterns

✕ À PROSCRIRE

- Stocker des objets gigantesques en localStorage (limite 5-10 MB selon nav).
- Cache sans TTL (« cache d'éternité » = bombe à retardement).
- Mettre des PII (email, token) dans des caches non sécurisés.
- Cache sur mutations (« optimisation prématurée » qui crée des incohérences).
- Oublier l'invalidation après changement de schéma → bumper APP_VERSION.

OBSERVABILITÉ

Mesure & monitoring

- Logger `cache_hit_rate` et `cache_miss_count` via PostHog.
- Lighthouse Performance pour mesurer LCP avec/sans cache.
- Network tab Chrome : voir « (from disk cache) » sur les bonnes ressources.

Sakata.com — Documentation interne · architecture-cache-strategy · v2026.4

« Moto oyo ayebi te kotombola miso na likolo, akoki te koyeba esika oyo mbula eutaka. » — Proverbe Sakata

SAKATA

© 2026 — Brume de la Rivière. Tous droits réservés.

CULTURE ECOLE LANGUE HISTOIRE FORUM